13. Discuss best practices in object-oriented design for developing layered systems. Analyze how.

Object-oriented design and layered architecture are commonly used together to develop software that is modular, maintainable and easier to understand. In a layered system, the application is divided into different layers, where each layer performs a specific responsibility.

Best Practices:-

1) Seperation of responsibilities: Each class and layer should have a clearly defined responsibility. For example, the presentation layer handles user interaction, while the business layer handles app. logic. This prevents different types of functionality from being mixed together.

2) Encapsulation: Data & related operations should be grouped inside classes, while unnecessary internal details are hidden. This protects data & reduces the impact of changes in one part of the system.

3. High Cohesion: Classes and modules should contain closely related functionality. High cohesion makes the code easier to understand, test & maintain.

4. Low coupling: Layers & classes should have minimum dependency on each other. Interfaces and abstraction can be used so that no component does not directly depend on the internal implementation of.

5. Use of Abstraction & interfaces: Interfaces allow different implementation to be used without changing the rest of the application. This improves flexibility and makes it easier to replace components.

6. Single Responsibility Principles: A class should have one primary responsibility. This prevents large, complicated classes & makes individual components easier to modify & test.

Reduction of complexity and improvement in code quality.

These practices divide a system into smaller components, making it easier to develop, test & maintain. Changes in one layer have minimal effect on others.

14. Layered Architecture and design Patterns in an Online banking system.

An online banking system handles sensitive operations such as login, account management, balance checking, fund transfer. Layered architecture helps seperate these functions & improve reliability and maintability.

1. Presentation layer: Provides the interface for customers to login, check, balances, perform transactions.
2. Business logic layer: Handles banking rules such as balance validation, transaction limit, authentication, fund transfer.
3. Data Access layer: Communicates with the database to retrieve & update customer, account, transaction information.
4. Database layer: Stores customer details, account balance & transaction records securely.

Design pattern can further improve the system : MVC seperate user interface and application layer. Factory pattern can create different transaction objects, while strategy pattern can support different payment or authentication methods.

Thus layered architecture and design pattern improves the banking system's reliability, security, maintainability, flexibility & scalability.

15. Integration of Design Principles, Layered Architecture & database systems.

Modern software system require proper organization to handle increasing complexity. Design principles, layered architecture, & database system work together to build reliable and scalable applicatio

Design principles such as encapsulation, abstraction, single responsibility, and dependency inversion help create modular and reusable classes. They reduce unnecessary dependencies & make code easier to maintain.

Layered architecture divide the application into presentation, business logic data access, database layers. Each layer has a specific responsibility reducing complexity and allowing changes to be made independly.

Database systems provide reliable storage and retrival of data. They maintain data consistency, security, transaction, backup, recovery.

When these concepts are integrated, the application becomes easier to develop test, maintain, scale. For example, in an e-commerce system, the presentation layer handles user, the business layer processes order, the data access layer communicates with the database, and database stores customers & order information.

Therefore, combining design principles, layred architecture & database systems helps create robust, maintainable, secure, scalable, software applications.